

CERTYFIKACJA

Mapa celów egzaminu

Exam guide z maja 2026 powiązany z modułami i laboratoriami

Spis treści

PRZEGLĄD	O egzaminie	3
SEKCJA 1	Databricks Intelligence Platform (6%)	4
SEKCJA 2	Data Ingestion and Loading (21%)	5
SEKCJA 3	Data Transformation and Modeling (22%)	8
SEKCJA 4	Working with Lakeflow Jobs (16%)	11
SEKCJA 5	Implementing CI/CD (10%)	13
SEKCJA 6	Troubleshooting, Monitoring, and Optimization (10%)	15
SEKCJA 7	Governance and Security (15%)	17
PRAKTYKA	Typowe schematy pytań	20
PLAN	Dwutygodniowy plan nauki	21

Ta mapa wiąże każdy cel z oficjalnego przewodnika egzaminacyjnego Databricks Certified Data Engineer Associate z modułami i laboratoriami szkolenia oraz podaje, co trzeba wiedzieć w każdym punkcie. Powtarzaj materiał sekcja po sekcji: otwórz notebooki wymienione w części *Gdzie na szkoleniu*, a potem sprawdź, czy bez notatek potrafisz wyjaśnić każdy punkt poniżej: to właśnie trzeba wiedzieć na egzamin. Tam, gdzie cel wykracza poza notebooki szkoleniowe, opis w tym dokumencie jest pełny i wystarcza do nauki.

PRZEGLĄD

O egzaminie

Element	Szczegóły
Wersja przewodnika	4 maja 2026
Pytania	45 ocenianych pytań wielokrotnego wyboru; mogą dojść pytania nieoceniane, a czas na nie jest doliczony
Czas i opłata	90 minut; 200 USD plus należne podatki
Forma	Online z nadzorem lub w centrum egzaminacyjnym; bez pomocy
Wymagania wstępne	Brak; zalecane szkolenie i około sześciu miesięcy praktyki
Ważność	Dwa lata; recertyfikacja przez zdanie aktualnej wersji egzaminu
Próg zaliczenia	Nie jest publikowany; celuj w co najmniej 80% w testach próbnych

Przewodniki egzaminacyjne zmieniają się co kilka miesięcy. Około dwóch tygodni przed egzaminem pobierz aktualną wersję z databricks.com/learn/certification i porównaj ją z tą mapą.

Sekcja	Domena	Waga	Ok. pytań	Dni szkolenia
1	Databricks Intelligence Platform	6%	3	Dzień 1
2	Data Ingestion and Loading	21%	9	Dni 1-2
3	Data Transformation and Modeling	22%	10	Dni 1-2 + samodzielna nauka
4	Working with Lakeflow Jobs	16%	7	Dzień 3
5	Implementing CI/CD	10%	4-5	Dzień 3
6	Troubleshooting, Monitoring, and Optimization	10%	4-5	Dni 2-3
7	Governance and Security	15%	7	Dzień 3

Liczby pytań to szacunek: waga pomnożona przez 45. Przewodnik nie numeruje celów; ta mapa numeruje je w kolejności z przewodnika (2.3 to trzeci cel sekcji 2) i skraca ich brzmienie. Odwołania wskazują notebooki demo (05_incremental_ingestion), laboratoria (lab_05_streaming, odpowiedzi i rozwiązania w notebooks/guides i notebooks/solution) oraz kod w materials/.

SEKCJA 1

Databricks Intelligence Platform (6%)

1.1 Główne komponenty: architektura, Delta Lake, Unity Catalog

Gdzie na szkoleniu: 00_intro, 01_platform_and_workspace, 03_delta_lake, lab_01_first_steps, lab_03_delta_dml

- Płaszczyzną sterowania (aplikacja webowa, harmonogram jobów, Unity Catalog) zarządza Databricks. Obliczenia działają jako serverless na koncie Databricks albo jako klasyczne zasoby w Twojej subskrypcji chmurowej. Dane pozostają w magazynie obiektowym.
- Delta Lake to pliki Parquet plus dziennik transakcji `_delta_log`. Zapewnia transakcje ACID, egzekwowanie schematu, `DESCRIBE HISTORY` do audytu oraz time travel przez `VERSION AS OF`, `TIMESTAMP AS OF` i `RESTORE`.
- Unity Catalog ma jeden metastore na region i przestrzeń nazw `catalog.schema.object`. Zarządza tabelami, widokami, wolumenami, funkcjami, modelami, połączeniami i external locations, rejestruje lineage i zdarzenia audytowe. Wolumeny (`/Volumes/...`) zastępują dla plików katalog główny DBFS i mounty.
- Scenariusz: wycofanie złego ładowania, ślad audytowy i jedno źródło prawdy dla BI i AI prowadzą do time travel w Delta Lake pod kontrolą Unity Catalog.

1.2 Usługi obliczeniowe: cechy, ograniczenia, modele kosztów

Gdzie na szkoleniu: 01_platform_and_workspace (zasoby obliczeniowe, drzewo decyzyjne), 08_job_orchestration (serverless dla jobów), lab_08_orchestration (zadanie końcowe)

- Wielu analityków z zapytaniami ad hoc, szybki start i kontrola kosztów: serverless SQL warehouse ze skalowaniem i automatycznym zatrzymaniem. Starsze pytania nazywają to klastrem high-concurrency z autoskalowaniem.
- Photon jest domyślnie włączony w SQL warehouse i na serverless. Tryby dostępu na klasycznych zasobach to standard (dawniej shared, wielu użytkowników) i dedicated (dawniej single user). Instance pools skracają start klastrów klasycznych.

Zasoby	Do czego	Model kosztów	Ograniczenia
Serverless: notebooki, joby, pipeline'y	Większość ETL i pracy interaktywnej; start w sekundy	DBU w czasie działania, bez osobnej opłaty za VM	Mało ustawialnych właściwości Sparka; brak init scripts i bibliotek klastra; query profile zamiast Spark UI
SQL warehouse (serverless)	SQL, dashboardy, BI, wielu jednoczesnych użytkowników	DBU w czasie działania; auto-stop	Tylko SQL; współbieżność przez dokładanie klastrów
Job compute (klasyczne)	Joby wymagające konkretnego runtime'u, bibliotek lub init scripts	Niższa stawka DBU plus VM; kończy się z uruchomieniem	Start trwa minuty
All-purpose compute (klasyczne)	Praca interaktywna	Wyższa stawka DBU plus VM	Bezczynny klaster kosztuje; ustaw auto-termination

SEKCJA 2

Data Ingestion and Loading (21%)

2.1 Wzorce i źródła ingestii

Gdzie na szkoleniu: 02_data_ingestion (metody ingestii, wzorce przetwarzania, `read_files()`, CTAS), 02a_lakeflow_connect_demo, 05_incremental_ingestion, lab_02_ingestion, lab_05_streaming

- **Batch** przy każdym uruchomieniu czyta wszystko (`spark.read`, CTAS na `read_files()`). **Przyrostowo** ładowane są tylko nowe dane, a stan przechowuje historia COPY INTO, checkpoint Auto Loadera lub streaming table. **Streaming** korzysta ze Structured Streaming; `trigger(availableNow=True)` przetwarza zaległości i kończy działanie.
- **Pliki lokalne** trafiają przez stronę *Add data* (do zarządzanej tabeli Delta) albo do wolumenu.
- **Konektory standardowe** (magazyn chmurowy przez Auto Loader, `read_files` lub COPY INTO; Kafka; SFTP) wymagają kodu pipeline'u, który utrzymujesz sam.
- **Konektory zarządzane** obsługują aplikacje SaaS (Salesforce, Workday, ServiceNow, Dynamics 365), bazy danych (SQL Server, MySQL, PostgreSQL, Oracle) i usługi plikowe (SharePoint, Google Drive). Konfiguruje się je w UI, API lub bundle'ach, a dane trafiają przyrostowo do streaming tables w Unity Catalog.

2.2 COPY INTO

Gdzie na szkoleniu: 05_incremental_ingestion (COPY INTO, idempotentność), lab_05_streaming zadanie 1

- Idempotentne: pliki już załadowane są pomijane, a `COPY_OPTIONS ('force' = 'true')` ładuje je ponownie. Pliki wybiera się przez `PATTERN` lub `FILES`.
- Źródłem jest wolumen lub URI chmurowe wewnątrz external location. Wymaga `MODIFY` na tabeli oraz `READ FILES` lub `READ VOLUME` na źródle.
- Dobre dla zespołów SQL i zaplanowanych ładowań do tysięcy plików. Przy milionach plików lub ciągłym napływie użyj Auto Loadera albo streaming table.

```
CREATE TABLE IF NOT EXISTS main.bronze.orders_raw;      -- schema comes from the files
COPY INTO main.bronze.orders_raw
FROM '/Volumes/main/landing/raw/orders/'
FILEFORMAT = JSON
FORMAT_OPTIONS ('mergeSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');
```

2.3 Auto Loader z egzekwowaniem i ewolucją schematu

Gdzie na szkoleniu: 05_incremental_ingestion (Auto Loader, ewolucja schematu i rescued data), lab_05_streaming zadania 2–6, BONUS_timestamps_autoloader_archive

- Schemat wymusisz przez `.schema(...)` lub `cloudFiles.schemaHints`. Bez `cloudFiles.inferColumnTypes` kolumny JSON i CSV są tekstowe.
- „Tryb wsadowy” to `availableNow=True` uruchamiane z zaplanowanego joba; checkpoint zapewnia przyrostowe przetwarzanie dokładnie raz. Każdy strumień potrzebuje własnego checkpointu.

- Wykrywanie plików: **directory listing** (domyślne, bez konfiguracji, listuje ścieżkę przy każdym uruchomieniu) albo **file notification** (zdarzenia, skaluje się do milionów plików). Zalecana forma to file events na external location i `cloudFiles.useManagedFileEvents = true`; forma klasyczna to `cloudFiles.useNotifications = true`.
- W SQL: `CREATE OR REFRESH STREAMING TABLE t AS SELECT * FROM STREAM read_files('/Volumes/...', format => 'json')`.

```
(spark.readStream.format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", "/Volumes/main/bronze/chk/orders_schema")
  .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
  .load("/Volumes/main/landing/raw/orders/")
  .writeStream.option("checkpointLocation", "/Volumes/main/bronze/chk/orders")
  .trigger(availableNow=True).toTable("main.bronze.orders"))
```

Tryb ewolucji	Gdy pojawią się nowe kolumny
addNewColumns (domyślny bez schematu)	Strumień się zatrzymuje, zapisany schemat jest aktualizowany, kolejne uruchomienie (np. retry joba) działa dalej
rescue	Schemat się nie zmienia; nieoczekiwane dane trafiają do <code>_rescued_data</code>
failOnNewColumns	Strumień kończy się błędem do czasu ręcznej zmiany schematu
none (domyślny ze schematem)	Nowe kolumny są ignorowane

2.4 Konfiguracja Lakeflow Connect

Gdzie na szkoleniu: 02_data_ingestion (Lakeflow Connect), 02a_lakeflow_connect_demo, przewodnik *Lakeflow Connect* (lakeflow_connect_sqlserver)

- Połączenie **Unity Catalog** przechowuje poświadczenia (do utworzenia `CREATE CONNECTION` na metastore, do użycia `USE CONNECTION`). **Pipeline ingestii** działa na serverless i zapisuje streaming tables (`USE CATALOG`, `USE SCHEMA`, `CREATE TABLE`).
- Konektory baz danych dodają **bramę ingestii** (ingestion gateway). Działa ciągle na klasycznych zasobach i buforuje snapshoty oraz zmiany w wolumenie, więc potrzebuje `CREATE VOLUME`.
- Ścieżka konfiguracji: **Data Ingestion**, potem konektor, połączenie, tabele źródłowe, miejsce docelowe i harmonogram. Działają też API, CLI i bundle'e.
- Przechwytywanie zmian: SQL Server po początkowym snapshotcie używa change tracking (tabele z kluczem głównym) albo CDC (tabele bez klucza); konektory SaaS używają cursorów API. Opcje to SCD typu 1 lub 2, wybór kolumn, nazwy tabel docelowych i pełne odświeżenie.

2.5 Klienci JDBC i REST w notebookach, orkestrowane jobami

Gdzie na szkoleniu: 02_data_ingestion (JDBC/ODBC), 08_job_orchestration (widżety i parametry), lab_08_orchestration. Wzorzec REST jest opisany tylko tutaj.

- Poświadczenia trzymaj w secret scopes; wartości z `dbutils.secrets.get` są maskowane w wynikach.
- Równoległy odczyt JDBC wymaga wszystkich czterech opcji: `partitionColumn`, `lowerBound`, `upperBound` i `numPartitions`; bez nich całą tabelę czyta jedno połączenie.

- JDBC sam w sobie nie jest przyrostowy: zapisuj znacznik (ostatnie `modified_at`) i używaj MERGE do upsertów. Surowe odpowiedzi REST zapisuj do wolumenu i ładuj Auto Loaderem.
- Orkestracja: zadanie notebooka, parametry joba, harmonogram i ponowienia dla przejściowych błędów sieci. Jeśli istnieje konektor zarządzany lub Lakehouse Federation, wybierz je.

```
orders = (spark.read.format("jdbc")
    .option("url", "jdbc:sqlserver://srv.database.windows.net:1433;database=sales")
    .option("dbtable", "dbo.orders")
    .option("user", dbutils.secrets.get("jdbc", "user"))
    .option("password", dbutils.secrets.get("jdbc", "password"))
    .option("partitionColumn", "order_id").option("lowerBound", 1)
    .option("upperBound", 5000000).option("numPartitions", 8)
    .load())
orders.write.mode("append").saveAsTable("main.bronze.orders_jdbc")

import requests
run_date = dbutils.widgets.get("run_date") # job parameter
resp = requests.get("https://api.example.com/v1/rates", params={"date": run_date},
    headers={"Authorization": f"Bearer {dbutils.secrets.get('api', 'token')}"},
    timeout=30)
resp.raise_for_status()
dbutils.fs.put(f"/Volumes/main/landing/raw/fx/{run_date}.json", resp.text, overwrite=True)
```

2.6 Wybór metody ingestii

Gdzie na szkoleniu: 02a_lakeflow_connect_demo (część A), 02_data_ingestion (porównanie metod ingestii), 05_incremental_ingestion

Kryteria: wolumen, częstotliwość i opóźnienie, typy danych, zarządzanie w Unity Catalog i koszt utrzymania. Zaczynaj od najbardziej zarządzanej opcji, a do własnego kodu przechodź dopiero wtedy, gdy ta opcja nie obsługuje źródła.

Wymaganie	Wybór
Pliki stale napływają do magazynu; schemat się zmienia	Auto Loader lub streaming table z <code>read_files</code>
Do tysięcy plików, zespół SQL, idempotentne przeładowania	COPY INTO
Aplikacja SaaS lub baza z konektorem zarządzanym	Konektor zarządzany Lakeflow Connect
Kafka lub inna magistrala komunikatów	Structured Streaming albo pipeline z konektorem Kafka
Brak konektora Databricks, jest licencja partnera	Narzędzie partnerskie (Fivetran, Qlik, Informatica)
Własne API lub nieobsługiwana baza, mały wolumen	Notebook z REST lub JDBC w jobie
Zapytania bez kopiowania danych	Lakehouse Federation

2.7 Dane półstrukturalne i niestukturalne przez konektory zarządzane

Gdzie na szkoleniu: 02_data_ingestion (opcje czytnika JSON), 05a_transformations (typy złożone), lab_06_transformations. Zarządzane konektory plikowe są opisane tylko tutaj.

- Zarządzane konektory SharePoint i Google Drive ładują BINARYFILE (jeden wiersz na plik z kolumną content i metadanymi; dla PDF, obrazów i dokumentów Office, później parsowanych np. przez `ai_parse_document`) albo

formaty strukturalne (CSV, JSON, XML, EXCEL, PARQUET, AVRO, ORC), parsowane do wierszy z trybami ewolucji schematu jak w Auto Loaderze.

- Konektory baz i SaaS mapują typy źródła na typy Delta (XML z SQL Server staje się `STRING`); przed spłaszczaniem sprawdź je przez `DESCRIBE TABLE`.
- Zagnieżdżony JSON można załadować jako jedną kolumnę `VARIANT`, odporną na zmiany schematu: służy do tego opcja `singleVariantColumn` w Auto Loaderze, `read_files` lub `COPY INTO`. Pola wyciąga się składnią `payload:customer.address.city::STRING`.
- Struktury spłaszcza się notacją kropkową. `explode` zamienia elementy tablicy na wiersze i pomija puste tablice; `explode_outer` je zachowuje. Teksty JSON parsuje się przez `from_json(col, schema)` lub `parse_json`. Surowe zagnieżdżenie zostaw w bronce, spłaszczaj w silver.

SEKCJA 3

Data Transformation and Modeling (22%)

Cele 3.1–3.5 są na tym szkoleniu do samodzielnej nauki: przerób `05a_transformations` i `lab_06_transformations` (w repozytorium są przewodnik i notebook z rozwiązaniem), a potem sprawdź się na punktach poniżej i na quizie z dnia 2. Cele 3.6 i 3.7 omawiamy na zajęciach w modułach 06 i 07.

3.1 Czyszczenie danych z bronce do silver

Gdzie na szkoleniu: `06_medallion_architecture` (warstwa silver), `materials/medallion/silver_*`, `02_data_ingestion` (transformacje `DataFrame`), `lab_06_transformations`

- Odczyt przez `spark.read.table("main.bronze.orders")`. Wartości `NULL` obsłużysz przez `dropna(subset=[...])`, `fillna({...})`, `coalesce()` lub `when().otherwise()`.
- Typy ujednolicaj przez `cast`, `to_date` i `to_timestamp` z jawnym formatem. Tryb ANSI (domyślny na serverless i w SQL warehouse) zgłasza błąd przy niepoprawnym rzutowaniu; `try_cast` zwraca wtedy `NULL`.
- Przed joinem lub deduplikacją normalizuj teksty przez `trim` i `lower`.
- Zapis przez `saveAsTable`, `CREATE OR REPLACE TABLE ... AS SELECT` albo `MERGE INTO` dla upsertów.

3.2 Joiny i unie

Gdzie na szkoleniu: `05a_transformations` (joiny), `lab_06_transformations` zadania 1–3, `lab_troubleshooting` zadanie 3, przewodnik `pyspark_vs_sparksql`

- Typy joinów: `inner`, `left`, `right`, `full`, `left_semi` (wiersze z dopasowaniem, kolumny lewej strony), `left_anti` (wiersze bez dopasowania), `cross`.
- Lista nazw kluczy daje po jednej kolumnie na klucz; wyrażenie typu `(a.k1 == b.k1) & (a.k2 == b.k2)` zostawia obie kopie, więc usuń je lub nadaj aliasy.
- Broadcast join widać w `explain()` jako `BroadcastHashJoin`; odpowiedź SQL to `/*+ BROADCAST(p) */`. Błędny lub obciążony klucz mnoży wiersze podobnie jak `cross join`.
- W PySpark `union()` i `unionAll()` działają tak samo: dopasowują kolumny po pozycji i zachowują duplikaty, więc do ich usunięcia dodaj `.distinct()`. W SQL `UNION` usuwa duplikaty, a `UNION ALL` je zachowuje.

```
from pyspark.sql.functions import broadcast
multi = sales.join(targets, on=["store_id", "sale_date"], how="inner") # multiple keys
left = orders.join(customers, "customer_id", "left")
fast = orders.join(broadcast(products), "product_id") # no shuffle of orders
grid = stores.crossJoin(calendar) # n x m rows
both = orders_2025.union(orders_2026) # by position, keeps duplicates
names = orders_2025.unionByName(orders_2026, allowMissingColumns=True)
```

3.3 Kolumny, wiersze i struktura tabel

Gdzie na szkoleniu: 02_data_ingestion (select, withColumn, cast, filter), 05a_transformations, lab_06_transformations zadanie 4, 03_delta_lake (ewolucja schematu)

- Kolumny: withColumn, withColumns({...}), withColumnRenamed, drop, select, selectExpr. Tekst dzieli się przez `F.split("name", " ").getItem(0)`.
- Wiersze: filter lub where z wyrażeniami kolumnowymi albo tekstem SQL.
- Tablice: explode, posexplode, collect_list, collect_set; funkcje wyższego rzędu transform, filter i exists działają bez rozwijania.
- Tabele: ALTER TABLE ... ADD COLUMNS. RENAME COLUMN i DROP COLUMN wymagają column mapping ('delta.columnMapping.mode' = 'name'). Zapis z mergeSchema dodaje kolumny.

3.4 Deduplikacja i agregacje

Gdzie na szkoleniu: 05a_transformations (deduplikacja, agregacje przybliżone i opisowe), lab_06_transformations zadania 3 i 5

- distinct() porównuje całe wiersze; dropDuplicates(["key"]) zostawia dowolny wiersz dla klucza. Najnowszy wiersz zachowasz przez row_number() albo QUALIFY w SQL.
- count("*") liczy wszystkie wiersze, a count(col) pomija NULL. approx_count_distinct (HyperLogLog++, domyślny błąd względny 5%) jest dużo tańsze od dokładnego zliczania unikalnych wartości.
- describe() zwraca count, mean, stddev, min i max; summary() dodaje percentyle 25%, 50% i 75%. W streamingu użyj dropDuplicatesWithinWatermark.

```
w = Window.partitionBy("order_id").orderBy(F.col("_ingest_ts").desc())
latest = df.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")
df.agg(F.countDistinct("customer_id"), F.approx_count_distinct("customer_id")).show()
```

3.5 Podstawowe parametry strojenia

Gdzie na szkoleniu: 05a_transformations (konfiguracje strojenia), lab_06_transformations zadanie 7, lab_troubleshooting zadanie 4

- Serverless przyjmuje tylko kilka właściwości, m.in. spark.sql.shuffle.partitions, spark.sql.session.timeZone, spark.sql.ansi.enabled i spark.sql.files.maxPartitionBytes. Ustawienie nieobsługiwanej właściwości kończy job błędem; zamiast tego stosuj odpowiedzi broadcast().
- Adaptive query execution (domyślnie włączone) w trakcie wykonania łączy partycje, przełącza na broadcast join i dzieli partycje z przekosem.

- Mierz ponownie, zmieniając jedną rzecz naraz na tych samych danych i zasobach. Zmierz czas akcji, np. `df.write.format("noop").mode("overwrite").save()`, porównaj rozmiar shuffle, liczbę zadań i spill, a potem przywróć ustawienie.

Parametr	Steruje	Domyślnie i uwagi
<code>spark.sql.shuffle.partitions</code>	Partycje po joinach i agregacjach	200 na klasycznych, auto na serverless; zmienny w trakcie; mniej dla małych danych
<code>spark.default.parallelism</code>	Domyślne partycje operacji RDD	Liczba rdzeni; shuffle DataFrame go nie używa
<code>spark.executor.memory</code>	Sterta executora	Ustawiany przed startem, zwykle przez typ węzła; za mało oznacza spill i OOM
<code>spark.driver.memory</code>	Sterta drivera	Stały po starcie; ważny przy <code>collect()</code> , <code>toPandas()</code> i broadcastach
<code>spark.sql.autoBroadcastJoinThreshold</code>	Największa tabela rozgłaszana automatycznie	10 MB; -1 wyłącza; zwiększ dla większych wymiarów

3.6 Obiekty gold: tabele, widoki, materialized views, streaming tables

Gdzie na szkoleniu: `07_lakeflow_pipelines` (typy tabel, `STREAM()` a bezpośrednie odwołanie), `06_medallion_architecture` (gold), `lab_07_lakeflow_pipeline` zadanie 4, `10_governance_and_security` (widoki dynamiczne)

- Streaming tables wymagają źródeł typu append-only, a materialized view czytający `STREAM(...)` jest niepoprawny. W Databricks SQL oba typy obiektów utrzymuje pipeline serverless.
- Wszystkie cztery są obiektami Unity Catalog; użytkownikom BI nadaj `SELECT`.

Obiekt	Przechowuje dane	Aktualność zapewnia	Zastosowanie
Tabela	Tak	Twoje zapisy: <code>INSERT</code> , <code>MERGE</code> , nadpisanie	Wymiary, SCD, pełna kontrola
Widok	Nie	Wykonanie zapytania przy odczycie	Lekka logika, warstwa bezpieczeństwa
Materialized view	Tak	Aktualizacja pipeline'u, <code>REFRESH</code> lub harmonogram; przyrostowo, gdy się da	Agregaty gold dla BI
Streaming table	Tak	Jednokrotne przetworzenie każdego nowego wiersza	Ingestia, warstwę dopisujące

```
CREATE MATERIALIZED VIEW main.gold.daily_revenue AS
SELECT order_date, SUM(total_amount) AS revenue FROM main.silver.orders GROUP BY order_date;
```

3.7 Kontrola jakości danych

Gdzie na szkoleniu: 07_lakeflow_pipelines (expectations, kwarantanna), lab_07_lakeflow_pipeline zadania 2, 6, 7, 03_delta_lake (constraints), 06_medallion_architecture

- Bez klauzuli `ON VIOLATION` wiersz zostaje, a naruszenie jest zapisywane jako metryka. W Pythonie dekoratory to `@dp.expect`, `@dp.expect_or_drop`, `@dp.expect_or_fail` oraz warianty `expect_all*`.
- Metryki widać w UI pipeline'u i w `event_log(TABLE(...))` w polu `details:flow_progress.data_quality.expectations`. Odrzucone wiersze nie są nigdzie zapisywane; jeśli są potrzebne, zapisz tabelę kwarantanny z odwróconą regułą.
- Ograniczenia Delta odrzucają cały zapis: `ALTER TABLE t ADD CONSTRAINT c CHECK (amount > 0)` lub `SET NOT NULL`.
- Sprawdzaj też unikalność kluczy, spójność referencyjną przez `left_anti` join, liczby wierszy między warstwami i świeżość danych.

```
CREATE OR REFRESH STREAMING TABLE silver_orders (
  CONSTRAINT valid_id EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT positive_amt EXPECT (total_amount > 0),           -- warn only
  CONSTRAINT known_status EXPECT (status IN ('NEW','PAID')) ON VIOLATION FAIL UPDATE
) AS SELECT * FROM STREAM(bronze_orders);
```

SEKCJA 4

Working with Lakeflow Jobs (16%)

4.1 Sterowanie przepływem: ponowienia, rozgałęzienia, pętle

Gdzie na szkoleniu: 08_job_orchestration (opcje i ponowienia; control flow jako kod: retries, If/else, For each), lab_08_orchestration zadania A, B, D

- **Ponowienia** ustawia się dla zadania (liczba, minimalny odstęp, ponowienie po timeoutcie). Pomagają przy błędach przejściowych, nie przy błędach w kodzie ani złych danych.
- **If/else condition** porównuje operandy przez `==`, `!=`, `>`, `>=`, `<`, `<=`. Operandami mogą być parametry joba lub task values, np. `{{tasks.validate.values.row_count}}`. Zadania podrzędne zależą od gałęzi `true` albo `false`.
- Task values: `dbutils.jobs.taskValues.set(key="row_count", value=n)` oraz `dbutils.jobs.taskValues.get(taskKey="validate", key="row_count", default=0, debugValue=0)`. `dbutils.notebook.exit()` nie tworzy task values.
- **For each** uruchamia zagnieżdżone zadanie dla każdego elementu tablicy JSON (dostępnego jako `{{input}}`), z ustawianą współbieżnością.
- **Run if:** *All succeeded* (domyślnie), *At least one succeeded*, *None failed*, *All done* (sprzątanie), *At least one failed* (alerty), *All failed*. **Repair run** uruchamia ponownie tylko zadania nieudane i pominięte oraz zależne od nich.

4.2 Typy zadań i zależności

Gdzie na szkoleniu: 08_job_orchestration (typy zadań, job wielozadaniowy, YAML joba medalionowego), lab_08_orchestration część 1, materials/cicd/resources/retailhub_job.yml

- W zakresie egzaminu: **Notebook**, **SQL** (zapytanie, alert lub plik na SQL warehouse), **Dashboard** (odświeża dashboard AI/BI i powiadamia subskrybentów) i **Pipeline** (uruchamia aktualizację pipeline'u). Inne to m.in. skrypt Python, wheel, JAR, Run Job i dbt.
- DAG definiuje `depends_on`; zadania, które nie zależą od siebie nawzajem, działają równolegle.
- Parametry joba trafiają do każdego zadania (`dbutils.widgets.get, {{job.parameters.name}}`); wartości dynamiczne to m.in. `{{job.run_id}}` i `{{job.start_time.iso_date}}`.
- Zasoby: dla notebooków domyślnie serverless; klaster joba może być wspólny dla kilku zadań; zadania SQL i dashboard wymagają SQL warehouse. Domyślnie aktywne jest jedno uruchomienie, a nadmiarowe są pomijane.

4.3 Harmonogramy i typy wyzwalaczy

Gdzie na szkoleniu: 08_job_orchestration (wyzwalacze i harmonogram, wyrażenia cron), lab_08_orchestration krok 7, zadanie C, zadanie końcowe, materials/orchestration/*_trigger_demo.py

- Cron w składni Quartz zaczyna się od sekund: `0 0 2 * * ?` to codziennie o 02:00, a `0 0 8 ? * MON-FRI` to dni robocze o 08:00.
- File arrival wymaga `READ FILES` na lokalizacji. Wyzwalacz table update przekazuje zadaniom `{{job.trigger.table_update.updated_tables}}`. Bundle'e w mode: development wstrzymują wyzwalacze.

Wyzwalacz	Uruchamia job, gdy	Ustawienia
Scheduled	Nadejdzie określony czas	Cron Quartz, strefa czasowa, wstrzymanie
File arrival	W wolumenie lub external location pojawią się nowe pliki	Minimalny odstęp, czekanie po ostatniej zmianie
Table update	Tabele Unity Catalog dostaną nowe commity	Dowolna lub wszystkie tabele, minimalny odstęp, czekanie po zmianie
Continuous	Zakończy się poprzednie uruchomienie	Wydłużanie przerw po błędach

4.4 Wyzwalacze czasowe czy sterowane danymi

Gdzie na szkoleniu: 08_job_orchestration (wyzwalacze), lab_08_orchestration krok 7 i zadanie C

- Scheduled**, gdy obowiązuje stały termin lub SLA albo źródło ładuje się o znanej porze; czas i koszt są przewidywalne.
- File arrival**, gdy pliki przychodzą nieregularnie i trzeba je szybko przetworzyć bez pustych uruchomień.
- Table update**, gdy job zależy od tabeli utrzymywanej przez inny pipeline lub zespół; job nie wystartuje przed gotowością danych.
- Job lub pipeline **continuous** do stałego przetwarzania niemal w czasie rzeczywistym. Kroki zależne od siebie umieszczaj jako zadania w jednym jobie, a nie jako osobne wyzwalacze.
- Częste dostawy grupuj ustawieniami *minimum time between triggers* i *wait after last change*; nie odpytuj danych w pętli w notebooku.

SEKCJA 5

Implementing CI/CD (10%)

5.1 Git folders: gałęzie, commit, push, pull requesty

Gdzie na szkoleniu: 09_cicd_and_automation (przejdźcie przez UI Git folders), lab_09_dabs (opcjonalny krok z Git folder), 01_platform_and_workspace

- Git folder tworzy się w **Workspace** przez **Create > Git folder**, podając URL repozytorium. Poświadczenia Git są w ustawieniach użytkownika, w sekcji kont powiązanych.
- **Okno Git** (przycisk gałęzi obok notebooka lub folderu) pozwala przełączać gałęzie, tworzyć je przez **Create Branch** od gałęzi bazowej, wykonać **Pull**, przejrzeć różnice, wpisać opis commita i wykonać **Commit & Push**. Merge, rebase i reset są w menu okna.
- Pull requesty otwiera się, recenzuje i scala u dostawcy Git (GitHub, Azure DevOps, GitLab) po wypchnięciu gałęzi. CI uruchamia potem `databricks bundle validate` i `deploy`.
- Każdy programista powinien mieć własny Git folder i gałąź: przełączenie gałęzi we wspólnym folderze przełącza ją wszystkim.

5.2 Zmienne bundle'a i nadpisanie targetów

Gdzie na szkoleniu: 09_cicd_and_automation (target overrides), lab_09_dabs zadania 1, 2, 6, materials/cicd/databricks.yml

- Do zmiennej odwołujesz się przez `${var.catalog}`, obok podstawień takich jak `${bundle.target}` i `${resources.pipelines.<key>.id}`. lookup ustala identyfikator po nazwie, np. warehouse'u.
- Pierwszeństwo, od najwyższego: `--var="catalog=x"`, `BUNDLE_VAR_catalog`, `.databricks/bundle/<target>/variable-overrides.json`, `variables` w targecie, na końcu `default`.
- Target nadpisuje mapowania najwyższego poziomu (`workspace`, `resources`, `run_as`, `presets`). `mode: development` dodaje do nazw prefiks `[dev <user>]` i wstrzymuje wyzwalacze; `mode: production` wymusza bezpieczne ustawienia.
- Zmienne są rozwiązywane przy wdrożeniu; wartości zmienne między uruchomieniami należą do parametrów `joba`.

```
variables:
  catalog:
    default: retailhub_dev
targets:
  dev:
    mode: development
    default: true
  prod:
    mode: production
    variables:
      catalog: retailhub_prod
    run_as:
      service_principal_name: "00000000-0000-0000-0000-000000000000"
```

5.3 Wdrażanie bundle'i

Gdzie na szkoleniu: 09_cicd_and_automation (struktura bundle'a, lista kontrolna wdrożenia), lab_09_dabs zadania 3–5, materials/cicd/

- Bundle to `databricks.yml` plus dołączane (`include`) pliki zasobów z definicjami jobów, pipeline'ów, dashboardów, schematów, wolumenów i innych obiektów, razem z ich kodem.
- `bundle deploy` wysyła pliki do katalogu głównego targetu, tworzy lub aktualizuje zasoby i śledzi stan. Zasoby usunięte z konfiguracji są kasowane przy następnym wdrożeniu.
- Ten sam commit promujesz przez `-t dev`, `-t test` i `-t prod`; różnią się tylko zmienne i nadpisanie. Odwołania typu `${resources.pipelines.retailhub_pipeline.id}` łączą zadania z pipeline'ami w każdym targecie.
- `bundle init` startuje z szablonu; `bundle generate job --existing-job-id <id>` i `bundle deployment bind` przejmują istniejące zasoby.

5.4 Databricks CLI w CI/CD

Gdzie na szkoleniu: 09_cicd_and_automation (instalacja i uwierzytelnianie CLI, integracja z CI), lab_09_dabs zadanie 3, materials/cicd/README.md

- Aktualne CLI to pojedynczy plik binarny (0.2xx lub nowszy); stary pakiet `pip databricks-cli` nie obsługuje bundle'i.
- CI uwierzytelnia się jako service principal przez OAuth M2M (`DATABRICKS_HOST`, `DATABRICKS_CLIENT_ID`, `DATABRICKS_CLIENT_SECRET`) lub workload identity federation. Akcja GitHub `databricks/setup-cli` instaluje CLI.
- Inne grupy poleceń zarządzają obiektami bezpośrednio: `databricks jobs`, `pipelines`, `fs`, `repos`.

Polecenie	Cel
<code>databricks auth login --host <url> --profile TRAINING</code>	Logowanie OAuth użytkownika zapisane jako profil
<code>databricks bundle validate -t prod</code>	Sprawdzenie konfiguracji i zmiennych; pierwszy krok w CI
<code>databricks bundle deploy -t prod</code>	Wysłanie plików, utworzenie lub aktualizacja zasobów
<code>databricks bundle run -t prod retailhub_job</code>	Uruchomienie joba lub pipeline'u z bundle'a
<code>databricks bundle summary / destroy</code>	Lista wdrożonych zasobów / ich usunięcie

SEKCJA 6

Troubleshooting, Monitoring, and Optimization (10%)

6.1 Historia uruchomień i punkty odniesienia

Gdzie na szkoleniu: `BONUS_troubleshooting` (trendy historii uruchomień, tabele systemowe), `08_job_orchestration` (monitoring przez tabele systemowe), `lab_08_orchestration` zadanie E

- **Jobs & Pipelines**, potem job i karta **Runs**: widok macierzy pokazuje czas całkowity i status każdego uruchomienia oraz czas każdego zadania; widok listy pokazuje start, czas trwania, wyzwalacz i wynik.
- Porównuj uruchomienie z punktem odniesienia, np. medianą ostatnich udanych uruchomień, a nie tylko z poprzednim. Stopniowy wzrost wskazuje na przyrost danych lub brak utrzymania układu plików. Skok po wdrożeniu wskazuje na tę zmianę. Pojedyncze odchylenia to często start zasobów, kolejka lub ponowienia.
- Progi ostrzeżeń o czasie trwania z powiadomieniami sygnalizują wolne uruchomienia. Długie trendy daje system. `lakeflow.job_run_timeline`; grupuj po `run_id`, bo długie uruchomienia zajmują kilka wierszy.

```
SELECT job_id, run_id, MIN(period_start_time) AS started,
       timestampdiff(SECOND, MIN(period_start_time), MAX(period_end_time)) AS duration_s,
       MAX(result_state) AS result_state
FROM system.lakeflow.job_run_timeline
WHERE period_start_time > current_timestamp() - INTERVAL 30 DAYS
GROUP BY job_id, run_id ORDER BY job_id, started DESC;
```

6.2 Kondycja pipeline'ów w UI jobów

Gdzie na szkoleniu: `lab_troubleshooting` zadania 1 i 5, `lab_08_orchestration` zadania D i E, `lab_07_lakeflow_pipeline` (monitoring)

- Stany uruchomień: Queued, Running, Succeeded, Failed, Timed out, Canceled, Skipped (osiągnięty limit współbieżności).
- W grafie zadań pierwsze nieudane zadanie to przyczyna. Zadania *Upstream failed* w ogóle nie wystartowały; zadania *Excluded* nie spełniły warunku Run if. Czytaj pierwsze linie błędu nieudanego zadania.
- Lista jobów pokazuje wyniki ostatnich uruchomień; tabele systemowe dają odsetek błędów w czasie. Dla pipeline'ów sprawdzaj graf, event log i metryki jakości.

6.3 Spark UI: przekos, shuffle, spill

Gdzie na szkoleniu: `BONUS_troubleshooting` (czytanie Spark UI), `04_delta_optimization` (wąskie gardła wydajności), `lab_troubleshooting` zadania 3 i 4

Otwórz **Spark UI**, potem **Stages** i *Summary Metrics* (min, 25. percentyl, mediana, 75. percentyl, max). Na serverless użyj query profile.

AQE dzieli partycję, która jest co najmniej pięć razy większa od mediany i większa niż 256 MB. Więcej executorów nie podzieli jednej za dużej partycji, a mniej partycji shuffle pogarsza przekos.

Problem	Objaw	Rozwiązanie
Przekos	Max czasu zadania i shuffle read dużo powyżej mediany	AQE skew join (domyślnie włączony); broadcast; salting gorącego klucza
Duży shuffle	Duże shuffle read i write na Exchange	Broadcast małych tabel; wczesne filtrowanie i wybór kolumn
Spill	<i>Spill (Memory)</i> i <i>Spill (Disk)</i> powyżej zera	Więcej partycji lub auto; węzły z większą pamięcią; usunięcie przekosu
Drobne zadania	Tysiące zadań trwających milisekundy	Mniej partycji lub auto; łączenie przez AQE

6.4 Liquid clustering i predictive optimization

Gdzie na szkoleniu: 04_delta_optimization (liquid clustering, predictive optimization), lab_04_optimization zadania 3 i 4

- Liquid clustering zastępuje partycjonowanie i Z-ordering (nie można ich łączyć), pozwala na maksymalnie cztery klucze i zmianę kluczy bez przepisywania danych. Wybieraj kolumny z filtrów i joinów.
- `CLUSTER BY AUTO` dobiera klucze na podstawie historii zapytań w zarządzanych tabelach Unity Catalog z predictive optimization.
- Predictive optimization uruchamia `OPTIMIZE`, `VACUUM` i `ANALYZE` na tabelach zarządzanych, ustawia się ją na poziomie katalogu lub schematu, jest domyślnie włączona dla nowych kont, a operacje zapisuje w `system.storage.predictive_optimization_operations_history`.

```
CREATE TABLE main.silver.orders CLUSTER BY (customer_id, order_date) AS SELECT * FROM
main.bronze.orders;
ALTER TABLE main.silver.orders CLUSTER BY AUTO;
OPTIMIZE main.silver.orders;                -- FULL reclusters all records
ALTER CATALOG main ENABLE PREDICTIVE OPTIMIZATION;
```

6.5 Nieudany start klastra, konflikty bibliotek, brak pamięci

Gdzie na szkoleniu: BONUS_troubleshooting (wzorce awarii), lab_troubleshooting zadanie 6, 01_platform_and_workspace (driver i executors)

Triage: zerowy czas zadań oznacza problem infrastruktury, więc sprawdź event log. Błąd przed jakimkolwiek etapem oznacza kod, uprawnienia lub ścieżki, więc czytaj pierwszą linię błędu. Wolne lub nieudane etapy oznaczają dane lub wydajność, więc sprawdź metryki etapów.

Problem	Objawy i gdzie patrzeć	Rozwiązanie
Zasoby nie startują	Błąd bez czasu zadań; błąd init script lub uruchomienia VM; Event log zasobów	Poprawić init script; zmienić rozmiar VM lub region; zwiększyć limit; instance pools; serverless
Konflikt bibliotek	ModuleNotFoundError lub AttributeError po zmianie biblioteki; błędy pip w event logu	Przypiąć wersje; %pip install pkg==x w notebooku i dbutils.library.restartPython(); środowiska serverless
OOM drivera	„Spark driver has stopped unexpectedly” po collect() lub toPandas()	Zapisywać do tabel; limit(); unikać dużych broadcastów; większy driver na końcu
OOM executora	OutOfMemoryError, utracone executory, spill; przekos lub eksplozja wierszy w planie	Usunąć przekos i poprawić klucze; więcej partycji; niższy próg broadcastu

SEKCJA 7

Governance and Security (15%)

7.1 Tabele zarządzane i zewnętrzne

Gdzie na szkoleniu: 10_governance_and_security (cykl życia tabel managed i external, konwersja), 03_delta_lake, BONUS_external_connection, przewodnik external_connection

- Tabele zarządzane trzymają dane w zarządzanym magazynie Unity Catalog i korzystają z predictive optimization. DROP TABLE można cofnąć przez UNDROP w ciągu 7 dni; potem pliki są usuwane.
- Tabele zewnętrzne używają ścieżki w external location (uprawnienie CREATE EXTERNAL TABLE). DROP TABLE usuwa tylko metadane, a pliki zostają.
- SET MANAGED wymaga własności tabeli i Runtime 17.3 LTS lub serverless; kopiuje dane do magazynu zarządzanego, zachowuje nazwę, historię i uprawnienia, a strumień trzeba potem uruchomić ponownie.
- Nie ma polecenia konwersji z managed na external: utwórz tabelę zewnętrzną przez CREATE TABLE ... LOCATION '...' AS SELECT, przepnij odbiorców i usuń tabelę zarządzaną. Obie tabele zmienia się przez ALTER TABLE ... RENAME TO, SET TBLPROPERTIES i SET OWNER TO.

```
CREATE TABLE main.silver.customers (id BIGINT, email STRING);           -- managed
CREATE TABLE main.bronze.orders_ext (id BIGINT, amount DECIMAL(10,2))  -- external
  LOCATION 'abfss://external@stdeatrainig.dfs.core.windows.net/orders';
ALTER TABLE main.bronze.orders_ext SET MANAGED;                       -- external to managed
ALTER TABLE main.bronze.orders_ext UNSET MANAGED;                     -- rollback within 14 days
```

7.2 GRANT, REVOKE i DENY

Gdzie na szkoleniu: 10_governance_and_security (zarządzanie dostępem, REVOKE i DENY), lab_10_governance zadania 1, 7, 9

- Dostęp jest domyślnie zabroniony. Odczyt tabeli wymaga SELECT na niej (lub na obiekcie nadrzędnym) oraz USE CATALOG i USE SCHEMA. Uprawnienia na katalogu lub schemacie obejmują obecne i przyszłe obiekty podrzędne; uprawnienia na metastore nie są dziedziczone.

- Podmioty: użytkownicy (e-mail), grupy konta i service principals (application ID), w odwrotnych apostrofach. Grupy lokalne workspace'u nie mogą dostawać uprawnień w Unity Catalog.
- Uprawnienia nadają właściciele, administratorzy metastore i posiadacze **MANAGE**. W UI: **Catalog Explorer**, obiekt, potem **Permissions**. **REVOKE** usuwa tylko to jedno nadanie; dostęp może pozostać przez grupę lub obiekt nadrzędny.
- **DENY** działa tylko w starych ACL tabel `hive_metastore`, gdzie ma pierwszeństwo przed **GRANT**. Unity Catalog nie ma **DENY** dla uprawnień do danych: odbierz uprawnienie tam, gdzie je nadano, nadawaj więcej albo użyj filtrów, masek i ABAC. Polityki ABAC **DENY** (Beta) mogą odebrać tylko **MANAGE ACCESS CONTROL**.

```
GRANT USE CATALOG ON CATALOG retail TO `analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA retail.gold TO `analysts`;
GRANT MODIFY ON TABLE retail.silver.orders TO `6f1b2c3d-1111-2222-3333-444455556666`; -- service principal
REVOKE SELECT ON SCHEMA retail.gold FROM `analysts`;
SHOW GRANTS ON SCHEMA retail.gold;
```

7.3 Maski kolumn i filtry wierszy

Gdzie na szkoleniu: 10_governance_and_security (natywne maski i filtry wierszy, widoki dynamiczne), lab_10_governance zadania 3–6, 8

- Funkcja filtra zwraca **BOOLEAN**; maska zwraca typ kolumny. Działają przy każdym zapytaniu, także przez widoki i także dla właściciela. Usuwa się je przez **DROP MASK** i **DROP ROW FILTER**.
- Tabela ma najwyżej jeden filtr wierszy, a kolumna najwyżej jedną maskę. Są widoczne w `information_schema.row_filters` i `information_schema.column_masks`.
- Widoki dynamiczne (`CASE WHEN is_account_group_member(...)`) to starsze podejście, chroniące tylko zapytania przez widok.

```
CREATE FUNCTION retail.sec.mask_email(email STRING)
RETURN CASE WHEN is_account_group_member('pii_readers') THEN email ELSE '***' END;
ALTER TABLE retail.silver.customers ALTER COLUMN email SET MASK retail.sec.mask_email;

CREATE FUNCTION retail.sec.region_filter(region STRING)
RETURN is_account_group_member('admins') OR region = 'EMEA';
ALTER TABLE retail.silver.orders SET ROW FILTER retail.sec.region_filter ON (region);
```

7.4 Polityki ABAC

Gdzie na szkoleniu: 10_governance_and_security (attribute-based access control)

- ABAC jest ogólnie dostępne od kwietnia 2026. **Governed tags** definiuje się na poziomie konta, z listą dozwolonych wartości i uprawnieniem **ASSIGN**. **Polityki** na katalogu, schemacie lub tabeli wiążą funkcję maski lub filtra z każdą pasującą kolumną lub tabelą, także utworzoną później.
- Klauzule: **TO** i **EXCEPT** wskazują podmioty; **WHEN has_tag_value(...)** wybiera tabelę; **MATCH COLUMNS ... AS alias** wybiera kolumny; **ON COLUMN** wskazuje maskowaną kolumnę; **USING COLUMNS (...)** przekazuje argumenty filtra. Starsza pisownia `hasTagValue` nadal działa.
- Jedna polityka zastępuje maski ustawiane tabela po tabeli. Opiekunowie danych tagują, a administratorzy bezpieczeństwa piszą polityki.

- Wymagania: serverless lub Runtime 16.4 i nowszy; **MANAGE** na obiekcie i **EXECUTE** na funkcji. Polityki sprawdzisz przez **SHOW POLICIES** i **DESCRIBE POLICY**.

```
ALTER TABLE retail.silver.customers ALTER COLUMN ssn SET TAGS ('pii' = 'ssn');
CREATE POLICY redact_ssn ON SCHEMA retail.silver
COLUMN MASK retail.sec.redact
TO `account users` EXCEPT `compliance`
FOR TABLES
MATCH COLUMNS has_tag_value('pii', 'ssn') AS ssn_col
ON COLUMN ssn_col;
```

PRAKTYKA

Typowe schematy pytań

Pytania egzaminacyjne to krótkie scenariusze z czterema odpowiedziami. Poniższe schematy parafrazują typowy styl pytań; nie są to pytania oficjalne.

Sygnal w pytaniu	Prawdopodobna odpowiedź	Dlaczego inne odpadają
Jedno zadanie dziesięć razy wolniejsze; max shuffle read dużo powyżej mediany	AQE skew join, broadcast lub salting	Więcej executorów nie dzieli partycji
Wycofanie złej ingestii, audyt, jedna kopia dla BI i AI	Time travel w Delta Lake pod Unity Catalog	Kopie plików nie są transakcyjne ani zarządzane
Wielu analityków naraz, szybki start, kontrola kosztów	Serverless SQL warehouse	Klasy jobów i jednowęzłowe nie pasują
Pliki stale napływają; schemat się zmienia	Auto Loader z schema location i trybem ewolucji	Zwykły odczyt ładuje wszystko od nowa
Salesforce lub SQL Server co godzinę, bez budżetu na kod	Konektor zarządzany Lakeflow Connect	JDBC to samodzielne budowanie CDC i ponowień
Ponowne uruchomienie nie może dublować danych, zespół SQL	COPY INTO	CTAS ładuje wszystkie pliki
Start tylko po zmianie tabeli źródłowej	Wyzwalacz table update	Harmonogram może ruszyć przed danymi
Ponowić tylko nieudaną część joba	Repair run	Pełne ponowienie powtarza udaną pracę
Ten sam kod w dev, test i prod z innymi katalogami	Zmienne i targety bundle'a, CLI w CI	Ręczne kopie nie są powtarzalne
Zerowy czas zadań, błąd init script	Event log zasobów	Kod się nie uruchomił, Spark UI jest pusty
Ukryć kolumnę poza jedną grupą w każdym zapytaniu	Maska kolumny z is_account_group_member	Widok chroni tylko zapytania przez siebie
Jedna reguła maskowania dla setek tabel z PII	Polityka ABAC na governed tags	Maski per tabela pomijają nowe tabele
Odebrać prawo odczytu w Unity Catalog	REVOKE tam, gdzie je nadano	DENY nie działa w Unity Catalog
Agregat dla dashboardu odświeżany według harmonogramu	Materialized view	Widok liczy się przy każdym zapytaniu

Gdy dwie odpowiedzi wydają się poprawne, wybierz opcję zarządzaną, deklaratywną i objętą governance, chyba że pytanie stawia wymaganie, którego ta opcja nie spełnia.

PLAN

Dwutygodniowy plan nauki

Około 1,5 godziny w dni robocze i 3 godziny w weekend. Jeśli w drugim tygodniu wyniki testów próbnych są poniżej 80%, przesunąć termin egzaminu.

Dzień	Zakres	Działania
1	Sekcje 1, 2.1-2.3	Przeczytać <code>01_platform_and_workspace</code> , <code>05_incremental_ingestion</code> ; powtórzyć <code>lab_05_streaming</code>
2	Sekcje 2.4-2.7	<code>02a_lakeflow_connect_demo</code> , przewodnik Lakeflow Connect; napisać notebook JDBC i REST
3	Sekcje 3.1-3.4	Powtórzyć <code>lab_06_transformations</code> bez podpowiedzi; joiny, unie, deduplikacja w PySpark i SQL
4	Sekcje 3.5-3.7	Jeden eksperyment strojenia z query profile; <code>lab_07_lakeflow_pipeline</code> zadania 2-7
5	Sekcja 4	Odtworzyć job z <code>lab_08_orchestration</code> : retries, If/else, For each, wyzwalacze, repair
6	Sekcje 5-6	Wdrożyć <code>materials/cicd</code> przez CLI; <code>lab_troubleshooting</code> , <code>BONUS_troubleshooting</code>
7	Sekcja 7	Powtórzyć <code>lab_10_governance</code> ; napisać politykę ABAC z pamięci; quizy szkoleniowe
8	Słabe punkty	Każdą błędną odpowiedź z quizów przypisać do celu i przeczytać go tutaj
9	Kursy oficjalne	Kursy self-paced Databricks Academy z przewodnika dla najsłabszych domen
10	Test próbny 1	90 minut z zegarem; przeanalizować każdą odpowiedź
11	Powtórka	Sekcje poniżej 80%; typowe schematy pytań
12	Praktyka	Jedno laboratorium na słabą domenę, w workspace lub Databricks Free Edition
13	Test próbny 2	Potwierdzić co najmniej 80%; sprawdzić aktualny przewodnik
14	Lekka powtórka	Tylko punkty z tej mapy; sprawdzić rezerwację i wymogi dokumentu tożsamości